

MAKALAH TUGAS PENGANTI UJIAN AKHIR SEMESTER
MATA KULIAH STRUKTUR DATA

Implementasi Struktur Data FIFO, LIFO, Tree, dan Graph dalam Simulasi Deteksi
Penipuan Transaksi Keuangan

Disusun untuk memenuhi tugas Ujian Akhir Semester mata kuliah Struktur Data

Dosen Pengampu:

Dede Abdurahman S.Kom.,M.MSi



Disusun oleh Kelompok 5:

Ahmad Gifariya	2514101003
Andika Hikmawan	2514101061
Dan Jago Anugrah	2514101012
Fian Abdullah Mirza	2514101005

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MAJALENGKA

2026

DAFTAR ISI

PEMBAGIAN TUGAS ANGGOTA	1
BAB I	2
PENDAHULUAN	2
1.1 Latar Belakang.....	2
1.2 Rumusan Masalah	2
1.3 Tujuan	3
BAB II.....	4
LANDASAN TEORI	4
2.1 Struktur Data	4
2.2 Queue dan Prinsip FIFO.....	4
2.3 Stack dan Prinsip LIFO	4
2.4 Tree	4
2.5 Graph	5
2.6 DFS Cycle Detection.....	5
2.7 Kompleksitas Big-O	5
BAB III	6
IMPLEMENTASI STRUKTUR DATA DAN MEKANISME PEMROSESAN	6
3.1 Gambaran Umum Program dan Pemodelan Data.....	6
3.2 Implementasi FIFO menggunakan Queue.....	7
3.3 Implementasi LIFO menggunakan Stack	8
3.4 Implementasi Tree untuk Klasifikasi Transaksi	10
3.5 Implementasi Graph dengan DFS Cycle Detection.....	12
3.6 Ringkasan Hasil Program	16
BAB IV	17
ANALISIS KOMPLEKSITAS	17
4.1 Kompleksitas Queue / FIFO.....	17
4.2 Kompleksitas Stack / LIFO	17
4.3 Kompleksitas Tree	18

4.4 Kompleksitas Graph	18
4.5 Kompleksitas Total Sistem	18
BAB V	20
PENUTUP	20
5.1 Kesimpulan.....	20
5.2 Saran	20
DAFTAR PUSTAKA	21

PEMBAGIAN TUGAS ANGGOTA

No	Nama Anggota	Tugas Utama	Materi yang Dikuasai
1	Ahmad Gifariya	Membahas proses transaksi masuk menggunakan Queue	FIFO
2	Andika Hikmawan	Membahas penyimpanan transaksi mencurigakan menggunakan Stack	LIFO
3	Dan Jago Anugrah	Membahas proses klasifikasi transaksi menggunakan Tree	Tree
4	Fian Abdullah Mirza	Membahas pemodelan hubungan antar akun menggunakan Graph	Graph

Pembagian tugas ini dibuat agar setiap anggota kelompok memiliki tanggung jawab pembahasan yang jelas. Setiap anggota memegang satu struktur data utama, yaitu FIFO, LIFO, Tree, dan Graph. Walaupun pembahasan dibagi per anggota, program yang dibuat tetap menjadi satu sistem simulasi yang saling terhubung.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan transaksi keuangan digital membuat proses transfer dana menjadi semakin cepat dan mudah. Namun, kemudahan tersebut juga dapat membuka peluang terjadinya penipuan transaksi keuangan, seperti transaksi mencurigakan, pemindahan dana antar banyak akun, transaksi nominal besar, dan pola aliran dana yang berputar.

Dalam mata kuliah Struktur Data, kasus tersebut dapat disimulasikan menggunakan beberapa struktur data, yaitu Queue, Stack, Tree, dan Graph. Queue digunakan untuk memproses transaksi berdasarkan prinsip FIFO. Stack digunakan untuk menyimpan transaksi mencurigakan berdasarkan prinsip LIFO. Tree digunakan untuk mengklasifikasikan status transaksi, sedangkan Graph digunakan untuk memodelkan hubungan antar akun.

Pada simulasi ini, akun direpresentasikan sebagai vertex, sedangkan transaksi direpresentasikan sebagai edge. Karena transaksi memiliki arah dari pengirim ke penerima dan memiliki nominal sebagai bobot, maka graph yang digunakan adalah directed weighted graph. Dengan pemodelan tersebut, sistem dapat mendeteksi pola transaksi mencurigakan, salah satunya siklus transaksi antar akun.

1.2 Rumusan Masalah

Rumusan masalah dalam makalah ini adalah sebagai berikut:

1. Bagaimana transaksi keuangan dimodelkan menggunakan Queue, Stack, Tree, dan Graph?
2. Bagaimana Queue dengan prinsip FIFO digunakan untuk memproses transaksi?
3. Bagaimana Stack dengan prinsip LIFO digunakan untuk menyimpan transaksi mencurigakan?
4. Bagaimana Tree digunakan untuk mengklasifikasikan status transaksi?
5. Bagaimana Graph digunakan untuk mendeteksi pola siklus transaksi?

6. Bagaimana kompleksitas waktu dan ruang dari struktur data yang digunakan?

1.3 Tujuan

Tujuan dari makalah ini adalah sebagai berikut:

1. Menjelaskan penerapan Queue, Stack, Tree, dan Graph dalam simulasi deteksi penipuan transaksi keuangan.
2. Membuat program Python yang memproses dan menganalisis data transaksi simulasi.
3. Menjelaskan pemodelan akun sebagai vertex dan transaksi sebagai edge.
4. Menjelaskan hasil deteksi siklus transaksi menggunakan DFS Cycle Detection.
5. Menganalisis kompleksitas waktu dan ruang menggunakan notasi Big-O.

BAB II

LANDASAN TEORI

2.1 Struktur Data

Struktur data adalah cara untuk menyimpan, mengatur, dan mengelola data agar dapat digunakan secara efisien dalam program. Struktur data membantu program dalam melakukan proses pencarian, penyimpanan, pengambilan, dan pengolahan data.

Dalam makalah ini, struktur data yang digunakan adalah Queue, Stack, Tree, dan Graph. Keempat struktur data tersebut diterapkan dalam satu sistem simulasi deteksi penipuan transaksi keuangan.

2.2 Queue dan Prinsip FIFO

Queue adalah struktur data yang menggunakan prinsip FIFO atau First In First Out. Artinya, data yang pertama kali masuk akan menjadi data pertama yang keluar atau diproses.

Dalam simulasi ini, Queue digunakan untuk menyimpan transaksi yang masuk sebelum diproses. Transaksi diproses berdasarkan urutan masuknya agar kronologi transaksi tetap sesuai.

Operasi dasar pada Queue adalah enqueue dan dequeue. Enqueue digunakan untuk menambahkan data ke dalam antrian, sedangkan dequeue digunakan untuk mengambil data dari bagian depan antrian.

2.3 Stack dan Prinsip LIFO

Stack adalah struktur data yang menggunakan prinsip LIFO atau Last In First Out. Artinya, data yang terakhir masuk akan menjadi data pertama yang keluar.

Dalam simulasi ini, Stack digunakan untuk menyimpan transaksi yang terdeteksi mencurigakan atau fraud. Dengan prinsip LIFO, transaksi mencurigakan yang paling baru ditemukan dapat diperiksa terlebih dahulu.

Operasi dasar pada Stack adalah push dan pop. Push digunakan untuk menambahkan data ke bagian atas Stack, sedangkan pop digunakan untuk mengambil data dari bagian atas Stack.

2.4 Tree

Tree adalah struktur data non-linear yang berbentuk hierarki. Tree terdiri dari node yang saling terhubung dengan hubungan parent dan child. Node paling atas disebut root, sedangkan node yang tidak memiliki child disebut leaf.

Dalam simulasi ini, Tree digunakan sebagai pohon keputusan sederhana untuk menentukan status transaksi. Status transaksi diklasifikasikan menjadi tiga kategori, yaitu aman, mencurigakan, dan fraud. Keputusan tersebut dibuat berdasarkan nominal transaksi dan keterlibatan transaksi dalam siklus graph.

2.5 Graph

Graph adalah struktur data non-linear yang terdiri dari vertex dan edge. Vertex adalah simpul, sedangkan edge adalah sisi yang menghubungkan antar simpul.

Dalam simulasi ini, akun direpresentasikan sebagai vertex dan transaksi direpresentasikan sebagai edge. Karena transaksi memiliki arah dari pengirim ke penerima, graph yang digunakan adalah directed graph. Selain itu, nominal transaksi digunakan sebagai bobot sehingga graph termasuk weighted graph.

Graph digunakan untuk menganalisis hubungan antar akun dan mendeteksi pola transaksi mencurigakan, seperti siklus transaksi. Siklus transaksi terjadi ketika dana berpindah dari satu akun ke akun lain dan akhirnya kembali ke akun awal.

2.6 DFS Cycle Detection

DFS atau Depth First Search adalah metode penelusuran graph yang dilakukan dengan cara menelusuri jalur sedalam mungkin sebelum kembali ke jalur lain. Dalam simulasi ini, DFS digunakan untuk mendeteksi siklus pada graph transaksi.

Siklus dalam graph transaksi dapat menjadi indikasi awal adanya pola transaksi mencurigakan. Contohnya, transaksi $A001 \rightarrow A002 \rightarrow A003 \rightarrow A001$ menunjukkan adanya aliran dana yang kembali ke akun awal melalui beberapa akun lain.

2.7 Kompleksitas Big-O

Kompleksitas Big-O digunakan untuk mengukur efisiensi suatu struktur data atau proses dalam program. Kompleksitas terbagi menjadi dua, yaitu time complexity dan space complexity.

Time complexity menunjukkan banyaknya waktu yang dibutuhkan program berdasarkan jumlah input, sedangkan space complexity menunjukkan banyaknya memori yang digunakan. Dalam makalah ini, kompleksitas digunakan untuk menganalisis Queue, Stack, Tree, dan Graph.

BAB III

IMPLEMENTASI STRUKTUR DATA DAN MEKANISME PEMROSESAN

3.1 Gambaran Umum Program dan Pemodelan Data

Program yang dibuat merupakan simulasi deteksi penipuan transaksi keuangan menggunakan bahasa Python. Program ini menggunakan data simulasi sebanyak 10 akun dan 15 transaksi. Dalam pemodelan graph, akun direpresentasikan sebagai vertex, sedangkan transaksi antar akun direpresentasikan sebagai edge.

Graph yang digunakan adalah directed weighted graph. Disebut directed karena setiap transaksi memiliki arah dari akun pengirim menuju akun penerima. Disebut weighted karena setiap transaksi memiliki bobot berupa nominal transaksi.

Data simulasi yang digunakan memenuhi ketentuan minimal 10 vertex dan 15 edge. Akun yang digunakan adalah A001 sampai A010, sedangkan transaksi yang digunakan adalah T01 sampai T15.

Dalam program ini, struktur data yang digunakan adalah Queue/FIFO, Stack/LIFO, Tree, dan Graph. Queue digunakan untuk memproses transaksi berdasarkan urutan masuk. Stack digunakan untuk menyimpan transaksi mencurigakan sebagai daftar investigasi. Tree digunakan untuk mengklasifikasikan status transaksi. Graph digunakan untuk memodelkan hubungan antar akun dan mendeteksi siklus transaksi.

Secara umum, alur program adalah sebagai berikut:

1. Data transaksi dimasukkan ke dalam Queue.
2. Graph dibentuk dari seluruh transaksi.
3. Graph dianalisis untuk mendeteksi siklus transaksi.
4. Transaksi diproses satu per satu dari Queue.
5. Tree menentukan status transaksi menjadi AMAN, MENCURIGAKAN, atau FRAUD.

6. Transaksi mencurigakan dan fraud dimasukkan ke dalam Stack.
7. Program menampilkan hasil akhir.

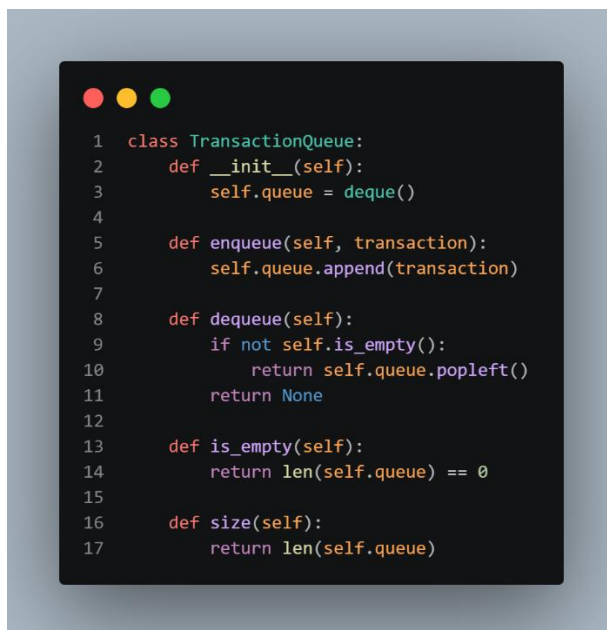
Berdasarkan hasil program, graph berhasil mendeteksi tiga siklus transaksi, yaitu A001 → A002 → A003 → A001, A004 → A005 → A006 → A004, dan A007 → A008 → A009 → A010 → A007. Siklus transaksi tersebut menjadi salah satu dasar dalam menentukan transaksi yang berstatus mencurigakan atau fraud.

3.2 Implementasi FIFO menggunakan Queue

FIFO atau First In First Out digunakan untuk memproses transaksi berdasarkan urutan masuk. Transaksi yang pertama kali masuk ke antrian akan diproses terlebih dahulu.

Dalam program ini, FIFO diterapkan menggunakan class TransactionQueue. Struktur data yang digunakan adalah deque dari library collections, karena deque mendukung proses penambahan dan pengambilan data secara efisien.

Contoh Kode FIFO



```
1 class TransactionQueue:
2     def __init__(self):
3         self.queue = deque()
4
5     def enqueue(self, transaction):
6         self.queue.append(transaction)
7
8     def dequeue(self):
9         if not self.is_empty():
10            return self.queue.popleft()
11        return None
12
13    def is_empty(self):
14        return len(self.queue) == 0
15
16    def size(self):
17        return len(self.queue)
```

Pada program utama, seluruh transaksi dimasukkan ke dalam Queue menggunakan method enqueue().

```
1 transaction_queue = TransactionQueue()
2
3 for transaction in transactions:
4     transaction_queue.enqueue(transaction)
5
6 print("\n=== FIFO / QUEUE ===")
7 print(f"Jumlah transaksi masuk ke queue: {transaction_queue.size()}")
8 print("Transaksi akan diproses berdasarkan urutan masuk.")
```

Output FIFO

```
=== FIFO / QUEUE ===
Jumlah transaksi masuk ke queue: 15
Transaksi akan diproses berdasarkan urutan masuk.
```

Pembahasan FIFO

Output tersebut menunjukkan bahwa terdapat 15 transaksi yang masuk ke dalam Queue. Transaksi akan diproses berdasarkan urutan masuknya, yaitu mulai dari T01 sampai T15.

Pada bagian klasifikasi, transaksi diproses menggunakan perulangan `while not transaction_queue.is_empty()`. Setiap transaksi diambil menggunakan method `dequeue()`. Karena `dequeue()` menggunakan `popleft()`, maka transaksi yang keluar adalah transaksi yang paling awal masuk. Hal ini membuktikan bahwa program menerapkan prinsip FIFO.

3.3 Implementasi LIFO menggunakan Stack

LIFO atau Last In First Out digunakan untuk menyimpan transaksi yang memiliki status MENCURIGAKAN atau FRAUD. Transaksi yang terakhir masuk ke Stack akan menjadi transaksi pertama yang ditampilkan dalam daftar investigasi. Dalam program ini, LIFO diterapkan menggunakan class `SuspiciousStack`.

Contoh Kode LIFO

```
1 class SuspiciousStack:
2     def __init__(self):
3         self.stack = []
4
5     def push(self, transaction, status, reason):
6         self.stack.append({
7             "transaction": transaction,
8             "status": status,
9             "reason": reason
10        })
11
12    def pop(self):
13        if not self.is_empty():
14            return self.stack.pop()
15        return None
16
17    def peek(self):
18        if not self.is_empty():
19            return self.stack[-1]
20        return None
21
22    def is_empty(self):
23        return len(self.stack) == 0
24
25    def size(self):
26        return len(self.stack)
```

Pada program utama, transaksi yang memiliki status MENCURIGAKAN atau FRAUD akan dimasukkan ke dalam Stack menggunakan method push().

```
1 if status in ["MENCURIGAKAN", "FRAUD"]:
2     suspicious_stack.push(current_transaction, status, reason)
```

Setelah semua transaksi selesai diproses, Stack ditampilkan menggunakan method pop().

```
1 print(
2     f"{current_transaction.id:<6} "
3     f"{current_transaction.sender:<8} "
4     f"{current_transaction.receiver:<8} "
5     f"{format_rupiah(current_transaction.amount):<18} "
6     f"{status:<15} "
7     f"{reason}"
8 )
```

Output LIFO

```
=== LIFO / STACK: DAFTAR INVESTIGASI TRANSAKSI MENCURIGAKAN ===
Transaksi terakhir yang masuk stack akan diperiksa terlebih dahulu.
```

Urutan	ID	Dari	Ke	Nominal	Status	Alasan
1	T10	A010	A007	Rp5.000.000	FRAUD	Nominal besar dan berada dalam siklus transaksi
2	T09	A009	A010	Rp5.100.000	FRAUD	Nominal besar dan berada dalam siklus transaksi
3	T08	A008	A009	Rp4.900.000	MENCURIGAKAN	Transaksi berada dalam siklus
4	T07	A007	A008	Rp5.000.000	FRAUD	Nominal besar dan berada dalam siklus transaksi
5	T06	A006	A004	Rp850.000	MENCURIGAKAN	Transaksi berada dalam siklus
6	T05	A005	A006	Rp900.000	MENCURIGAKAN	Transaksi berada dalam siklus
7	T04	A004	A005	Rp800.000	MENCURIGAKAN	Transaksi berada dalam siklus
8	T03	A003	A001	Rp1.600.000	MENCURIGAKAN	Transaksi berada dalam siklus
9	T02	A002	A003	Rp1.750.000	MENCURIGAKAN	Transaksi berada dalam siklus
10	T01	A001	A002	Rp1.500.000	MENCURIGAKAN	Transaksi berada dalam siklus

Pembahasan LIFO

Output tersebut menunjukkan bahwa transaksi T10 muncul pada urutan pertama dalam daftar investigasi. Hal ini terjadi karena T10 adalah transaksi mencurigakan terakhir yang dimasukkan ke Stack.

Setelah T10, data yang keluar berikutnya adalah T09, T08, T07, dan seterusnya sampai T01. Urutan ini membuktikan bahwa Stack bekerja menggunakan prinsip LIFO, yaitu data terakhir yang masuk akan menjadi data pertama yang keluar.

Dalam simulasi ini, Stack berfungsi sebagai daftar investigasi transaksi mencurigakan. Transaksi yang paling baru terdeteksi dapat diperiksa terlebih dahulu.

3.4 Implementasi Tree untuk Klasifikasi Transaksi

Tree digunakan sebagai pohon keputusan sederhana untuk menentukan status transaksi. Status transaksi dibagi menjadi tiga kategori, yaitu AMAN, MENCURIGAKAN, dan FRAUD.

Aturan keputusan yang digunakan adalah sebagai berikut:

1. Jika nominal transaksi lebih dari atau sama dengan Rp5.000.000 dan transaksi berada dalam siklus, maka statusnya FRAUD.

2. Jika nominal transaksi lebih dari atau sama dengan Rp5.000.000 tetapi tidak berada dalam siklus, maka statusnya MENCURIGAKAN.
3. Jika transaksi berada dalam siklus, maka statusnya MENCURIGAKAN.
4. Jika nominal transaksi lebih dari atau sama dengan Rp4.500.000, maka statusnya MENCURIGAKAN.
5. Jika tidak memenuhi kondisi tersebut, maka statusnya AMAN.

Contoh Kode Tree

```
1 class DecisionTree:
2     def __init__(self):
3         self.high_amount_threshold = 5_000_000
4         self.warning_amount_threshold = 4_500_000
5
6     def classify(self, transaction, cycle_edges):
7         """
8         Pohon keputusan sederhana:
9
10        1. Jika nominal >= 5.000.000 dan transaksi masuk siklus:
11            status = FRAUD
12
13        2. Jika transaksi masuk siklus:
14            status = MENCURIGAKAN
15
16        3. Jika nominal >= 4.500.000:
17            status = MENCURIGAKAN
18
19        4. Selain itu:
20            status = AMAN
21        """
22
23        edge = (transaction.sender, transaction.receiver)
24        is_cycle_edge = edge in cycle_edges
25
26        # Root: cek nominal besar
27        if transaction.amount >= self.high_amount_threshold:
28            # Cabang: cek apakah transaksi berada dalam siklus graph
29            if is_cycle_edge:
30                return "FRAUD", "Nominal besar dan berada dalam siklus transaksi"
31            else:
32                return "MENCURIGAKAN", "Nominal besar"
33
34        # Cabang berikutnya: cek apakah transaksi berada dalam siklus graph
35        if is_cycle_edge:
36            return "MENCURIGAKAN", "Transaksi berada dalam siklus"
37
38        # Cabang berikutnya: cek nominal mendekati besar
39        if transaction.amount >= self.warning_amount_threshold:
40            return "MENCURIGAKAN", "Nominal transaksi mendekati batas risiko"
41
42        # Leaf terakhir: aman
43        return "AMAN", "Tidak masuk pola risiko utama"
44
```

Output Klasifikasi Tree

```
=== PROSES TRANSAKSI FIFO + KLASIFIKASI TREE ===
```

ID	Dari	Ke	Nominal	Status	Alasan
T01	A001	A002	Rp1.500.000	MENCURIGAKAN	Transaksi berada dalam siklus
T02	A002	A003	Rp1.750.000	MENCURIGAKAN	Transaksi berada dalam siklus
T03	A003	A001	Rp1.600.000	MENCURIGAKAN	Transaksi berada dalam siklus
T04	A004	A005	Rp800.000	MENCURIGAKAN	Transaksi berada dalam siklus
T05	A005	A006	Rp900.000	MENCURIGAKAN	Transaksi berada dalam siklus
T06	A006	A004	Rp850.000	MENCURIGAKAN	Transaksi berada dalam siklus
T07	A007	A008	Rp5.000.000	FRAUD	Nominal besar dan berada dalam siklus transaksi
T08	A008	A009	Rp4.900.000	MENCURIGAKAN	Transaksi berada dalam siklus
T09	A009	A010	Rp5.100.000	FRAUD	Nominal besar dan berada dalam siklus transaksi
T10	A010	A007	Rp5.000.000	FRAUD	Nominal besar dan berada dalam siklus transaksi
T11	A001	A004	Rp300.000	AMAN	Tidak masuk pola risiko utama
T12	A002	A005	Rp250.000	AMAN	Tidak masuk pola risiko utama
T13	A003	A006	Rp280.000	AMAN	Tidak masuk pola risiko utama
T14	A007	A002	Rp700.000	AMAN	Tidak masuk pola risiko utama
T15	A009	A003	Rp650.000	AMAN	Tidak masuk pola risiko utama

Pembahasan Tree

Berdasarkan output tersebut, Tree berhasil mengklasifikasikan transaksi menjadi tiga status. Transaksi T01 sampai T06 dikategorikan MENCURIGAKAN karena transaksi tersebut berada dalam siklus. Transaksi T07, T09, dan T10 dikategorikan FRAUD karena nominalnya besar dan berada dalam siklus transaksi.

Sementara itu, transaksi T11 sampai T15 dikategorikan AMAN karena tidak berada dalam edge siklus dan nominalnya tidak melewati batas risiko. Dengan demikian, Tree berfungsi sebagai mekanisme pengambilan keputusan berdasarkan kondisi tertentu.

3.5 Implementasi Graph dengan DFS Cycle Detection

Graph digunakan untuk memodelkan hubungan antar akun. Dalam program ini, akun menjadi vertex, transaksi menjadi edge, dan nominal transaksi menjadi weight. Karena transaksi memiliki arah dari pengirim ke penerima, graph yang digunakan adalah directed weighted graph.

Graph direpresentasikan menggunakan adjacency list. Representasi ini dipilih karena lebih mudah digunakan untuk menyimpan daftar transaksi keluar dari setiap akun.

Contoh Kode Graph

```
1 class TransactionGraph:
2     def __init__(self):
3         self.adj_list = defaultdict(list)
4         self.vertices = set()
5
6     def add_transaction(self, transaction):
7         sender = transaction.sender
8         receiver = transaction.receiver
9         amount = transaction.amount
10
11         self.vertices.add(sender)
12         self.vertices.add(receiver)
13
14         # Directed edge: sender -> receiver
15         self.adj_list[sender].append((receiver, amount, transaction.id))
16
17     def display_graph(self):
18         print("\n=== REPRESENTASI GRAPH: ADJACENCY LIST ===")
19
20         for account in sorted(self.vertices):
21             edges = self.adj_list.get(account, [])
22
23             if edges:
24                 edge_text = []
25                 for receiver, amount, transaction_id in edges:
26                     edge_text.append(
27                         f"{receiver}({transaction_id}, {format_rupiah(amount)})"
28                     )
29                 print(f"{account} -> {' '.join(edge_text)}")
30             else:
31                 print(f"{account} -> Tidak ada transaksi keluar")
```

Untuk mendeteksi siklus transaksi, program menggunakan method `detect_cycles()`. Siklus terjadi ketika terdapat jalur transaksi yang kembali ke akun awal.

Contoh Kode DFS Cycle Detection

```
1  def detect_cycles(self):
2      """
3      Mendeteksi siklus pada directed graph.
4      Siklus berarti terdapat jalur transaksi yang kembali ke akun awal.
5      """
6
7      cycles_set = set()
8
9      def normalize_cycle(cycle):
10         """
11         Normalisasi siklus supaya siklus yang sama tidak tercatat berulang.
12         Contoh:
13         A001 -> A002 -> A003
14         A002 -> A003 -> A001
15         dianggap siklus yang sama.
16         """
17         min_index = min(range(len(cycle)), key=lambda i: cycle[i])
18         normalized = cycle[min_index:] + cycle[:min_index]
19         return tuple(normalized)
20
21     def dfs(start, current, path, visited):
22         for neighbor, amount, transaction_id in self.adj_list.get(current, []):
23             # Jika kembali ke akun awal, berarti ditemukan siklus
24             if neighbor == start and len(path) >= 2:
25                 cycle = normalize_cycle(path.copy())
26                 cycles_set.add(cycle)
27
28             # Jika neighbor belum dikunjungi di jalur ini, lanjut DFS
29             elif neighbor not in visited:
30                 visited.add(neighbor)
31                 path.append(neighbor)
32
33                 dfs(start, neighbor, path, visited)
34
35                 path.pop()
36                 visited.remove(neighbor)
37
38     for start in sorted(self.vertices):
39         dfs(start, start, [start], {start})
40
41     cycles = [list(cycle) for cycle in cycles_set]
42     cycles.sort(key=lambda c: (len(c), c))
43
44     return cycles
45
46 def get_cycle_edges(self, cycles):
47     """
48     Mengambil pasangan edge yang termasuk dalam siklus.
49     Contoh siklus:
50     A001 -> A002 -> A003 -> A001
51
52     Edge siklus:
53     (A001, A002), (A002, A003), (A003, A001)
54     """
55     cycle_edges = set()
56
57     for cycle in cycles:
58         for i in range(len(cycle)):
59             sender = cycle[i]
60             receiver = cycle[(i + 1) % len(cycle)]
61             cycle_edges.add((sender, receiver))
62
63     return cycle_edges
```

Output Representasi Graph

```
=== REPRESENTASI GRAPH: ADJACENCY LIST ===
A001 -> A002(T01, Rp1.500.000), A004(T11, Rp300.000)
A002 -> A003(T02, Rp1.750.000), A005(T12, Rp250.000)
A003 -> A001(T03, Rp1.600.000), A006(T13, Rp280.000)
A004 -> A005(T04, Rp800.000)
A005 -> A006(T05, Rp900.000)
A006 -> A004(T06, Rp850.000)
A007 -> A008(T07, Rp5.000.000), A002(T14, Rp700.000)
A008 -> A009(T08, Rp4.900.000)
A009 -> A010(T09, Rp5.100.000), A003(T15, Rp650.000)
A010 -> A007(T10, Rp5.000.000)
```

Output Deteksi Siklus Graph

```
=== HASIL DETEKSI SIKLUS GRAPH ===
Siklus 1: A001 -> A002 -> A003 -> A001
Siklus 2: A004 -> A005 -> A006 -> A004
Siklus 3: A007 -> A008 -> A009 -> A010 -> A007
```

Pembahasan Graph

Output adjacency list menunjukkan hubungan antar akun berdasarkan transaksi yang terjadi. Contohnya, A001 memiliki transaksi keluar ke A002 melalui T01 dan ke A004 melalui T11.

Dari hasil DFS Cycle Detection, ditemukan tiga siklus transaksi. Siklus pertama adalah A001 → A002 → A003 → A001. Siklus kedua adalah A004 → A005 → A006 → A004. Siklus ketiga adalah A007 → A008 → A009 → A010 → A007.

Siklus transaksi dapat menjadi indikasi awal transaksi mencurigakan karena dana bergerak melalui beberapa akun dan kembali ke akun awal. Dalam sistem ini, edge yang termasuk ke dalam siklus digunakan sebagai dasar untuk menentukan apakah transaksi masuk kategori MENCURIGAKAN atau FRAUD.

3.6 Ringkasan Hasil Program

Setelah seluruh proses dijalankan, program menghasilkan ringkasan sebagai berikut.

```
=== RINGKASAN HASIL AKHIR ===  
Total transaksi           : 15  
Total akun / vertex       : 10  
Total transaksi / edge    : 15  
Total siklus terdeteksi   : 3  
Transaksi aman            : 5  
Transaksi mencurigakan    : 7  
Transaksi fraud           : 3
```

Berdasarkan ringkasan tersebut, program memproses 15 transaksi yang melibatkan 10 akun. Graph yang terbentuk memiliki 10 vertex dan 15 edge. Program berhasil mendeteksi 3 siklus transaksi. Dari hasil klasifikasi, terdapat 5 transaksi AMAN, 7 transaksi MENCURIGAKAN, dan 3 transaksi FRAUD.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Kompleksitas Queue / FIFO

Operasi utama pada Queue adalah enqueue dan dequeue.

Operasi	Kompleksitas Waktu	Kompleksitas Ruang
Enqueue	$O(1)$	$O(n)$
Dequeue	$O(1)$	$O(n)$
Memproses seluruh transaksi	$O(n)$	$O(n)$

Keterangan:

n adalah jumlah transaksi. Karena setiap transaksi dimasukkan satu kali dan diproses satu kali, kompleksitas total pemrosesan Queue adalah $O(n)$. Penggunaan ruang juga $O(n)$ karena Queue menyimpan sejumlah transaksi yang akan diproses.

4.2 Kompleksitas Stack / LIFO

Operasi utama pada Stack adalah push dan pop.

Operasi	Kompleksitas Waktu	Kompleksitas Ruang
Push	$O(1)$	$O(s)$
Pop	$O(1)$	$O(s)$
Menampilkan seluruh transaksi mencurigakan	$O(s)$	$O(s)$

Keterangan:

s adalah jumlah transaksi mencurigakan. Operasi push dan pop pada Stack memiliki kompleksitas $O(1)$. Penggunaan ruang bergantung pada jumlah transaksi mencurigakan yang disimpan.

4.3 Kompleksitas Tree

Tree digunakan untuk proses klasifikasi transaksi. Setiap transaksi melewati beberapa kondisi dalam pohon keputusan.

Proses	Kompleksitas Waktu	Kompleksitas Ruang
Klasifikasi satu transaksi	$O(h)$	$O(1)$
Klasifikasi seluruh transaksi	$O(n \times h)$	$O(1)$

Keterangan:

h adalah tinggi Tree, sedangkan n adalah jumlah transaksi. Karena Tree yang digunakan dalam simulasi ini sederhana dan jumlah kondisinya terbatas, proses klasifikasi dapat berjalan dengan efisien. Jika tinggi Tree dianggap tetap, maka klasifikasi seluruh transaksi dapat dianggap mendekati $O(n)$.

4.4 Kompleksitas Graph

Graph direpresentasikan menggunakan adjacency list. Proses yang dianalisis adalah pembuatan graph dan DFS Cycle Detection.

Proses	Kompleksitas Waktu	Kompleksitas Ruang
Membuat adjacency list	$O(V + E)$	$O(V + E)$
DFS Cycle Detection	$O(V + E)$	$O(V)$
Menampilkan hasil siklus	$O(c)$	$O(c)$

Keterangan:

V adalah jumlah vertex atau akun. E adalah jumlah edge atau transaksi. c adalah jumlah siklus yang ditemukan.

DFS Cycle Detection memiliki kompleksitas $O(V + E)$ karena algoritma mengunjungi setiap vertex dan edge paling banyak satu kali.

4.5 Kompleksitas Total Sistem

Secara umum, kompleksitas sistem dapat ditulis sebagai:

$$O(n) + O(n \times h) + O(V + E)$$

Karena n adalah jumlah transaksi dan E juga merepresentasikan jumlah transaksi pada graph, maka kompleksitas sistem dapat disederhanakan menjadi:

$$O(n \times h + V + E)$$

Jika tinggi Tree dianggap kecil dan tetap, maka kompleksitas sistem mendekati:

$$O(V + E)$$

Hal ini menunjukkan bahwa sistem simulasi cukup efisien untuk menggambarkan penerapan struktur data dalam analisis transaksi keuangan.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan, dapat disimpulkan bahwa struktur data FIFO, LIFO, Tree, dan Graph dapat diterapkan secara terpadu dalam simulasi deteksi penipuan transaksi keuangan.

FIFO digunakan untuk memproses transaksi berdasarkan urutan masuk sehingga sistem tetap mengikuti kronologi transaksi. LIFO digunakan untuk menyimpan riwayat transaksi mencurigakan agar transaksi terbaru dapat diperiksa terlebih dahulu. Tree digunakan untuk membuat aturan keputusan sederhana dalam menentukan status transaksi. Graph digunakan untuk memodelkan hubungan antar akun dan mendeteksi pola transaksi mencurigakan, terutama pola siklus antar akun.

Dalam studi kasus ini, akun direpresentasikan sebagai vertex, sedangkan transaksi direpresentasikan sebagai edge. Graph yang digunakan adalah directed weighted graph karena transaksi memiliki arah dan bobot berupa nominal transaksi. Dengan menggunakan DFS Cycle Detection, sistem dapat menemukan pola transaksi berputar seperti $A001 \rightarrow A002 \rightarrow A003 \rightarrow A001$.

Dari analisis kompleksitas, Queue dan Stack memiliki operasi dasar $O(1)$, Tree memiliki kompleksitas $O(h)$ untuk satu transaksi, sedangkan DFS pada Graph memiliki kompleksitas $O(V + E)$. Dengan demikian, sistem simulasi ini cukup efisien untuk menggambarkan penerapan struktur data dalam analisis transaksi keuangan.

5.2 Saran

Sistem yang dibuat masih berupa simulasi sederhana. Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menggunakan dataset transaksi yang lebih besar, menambahkan atribut transaksi seperti waktu, lokasi, perangkat, dan jenis transaksi, serta menggunakan metode analisis yang lebih kompleks seperti machine learning atau Graph Neural Network.

DAFTAR PUSTAKA

- Cheng, D., Zou, Y., Xiang, S., & Jiang, C. (2024). Graph neural networks for financial fraud detection: A review. *Frontiers of Computer Science*, 0(0), 1–17. <https://doi.org/10.1007/s11704-024-40474-y>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley.
- Kurshan, E., & Shen, H. (2021). *Graph computing for financial crime and fraud detection: Trends, challenges and outlook*. arXiv. <https://arxiv.org/abs/2103.03227>
- Pourhabibi, T., Ong, K. L., Kam, B. H., & Boo, Y. L. (2020). Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decision Support Systems*, 133, 113303. <https://doi.org/10.1016/j.dss.2020.113303>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.